

# Software Re-Engineering

---

Week 5

# Refactoring Techniques, Catalogs & Tools

# Refactoring

- It refers to the process of restructuring existing code without changing its external behavior.
- It is aimed at improving the internal structure of the code, making it more readable, maintainable, and efficient.
  - Enhances code readability.
  - Simplifies complexity.
  - Makes the system more flexible and adaptable to change.
  - Facilitates bug fixing and feature additions.
  - Reduces technical debt.

# Principles of Refactoring



Preserve  
Behaviour



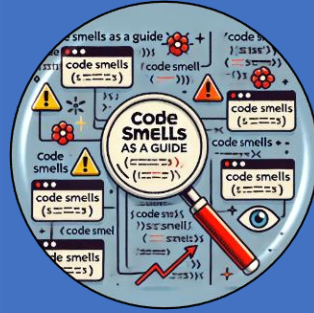
Small,  
Incremental  
Changes



Continuous  
Process



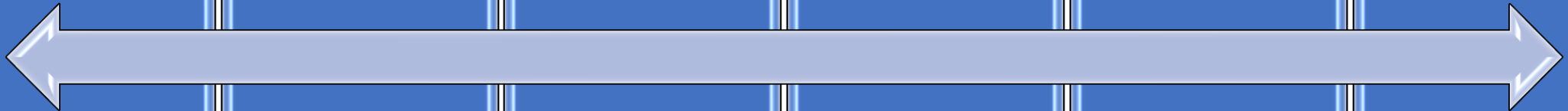
Test-Driven  
Refactoring



Code Smells  
as a Guide



Don't  
Refactor  
Without  
Reason



# Preserve Behavior

- Refactoring should never change the external behavior of the software.
- The core functionality, user experience, and outputs should remain consistent, even as the internal structure of the code is improved.
- While refactoring focuses on improving the internal code quality (readability, performance, maintainability), users and stakeholders expect the system to behave the same as it did before.

# Small, Incremental Changes

- Refactoring should be done in small, manageable steps, each of which can be tested individually to confirm that behavior is preserved.
- Making large changes to the code all at once increases the risk of introducing errors or breaking functionality.
- Small changes allow for quicker identification of issues and make it easier to revert if something goes wrong.

# Continuous Process

- Refactoring is an ongoing process that should be integrated into regular development, rather than being a one-time effort. It is part of maintaining a healthy source code.
- Over time, as new features are added or bugs are fixed, the code can become more complex.
- Regular refactoring helps to reduce technical debt and keep the source code maintainable and scalable.

# Test-Driven Refactoring

- Refactoring should always be done with test cases in place. Unit tests, integration tests, and automated tests ensure that the refactored code behaves the same way as before.
- Test cases provide a safety net. Before refactoring, test cases are written to verify the existing functionality.
- After refactoring, these test cases can be run to confirm that the changes haven't introduced any bugs or broken functionality.



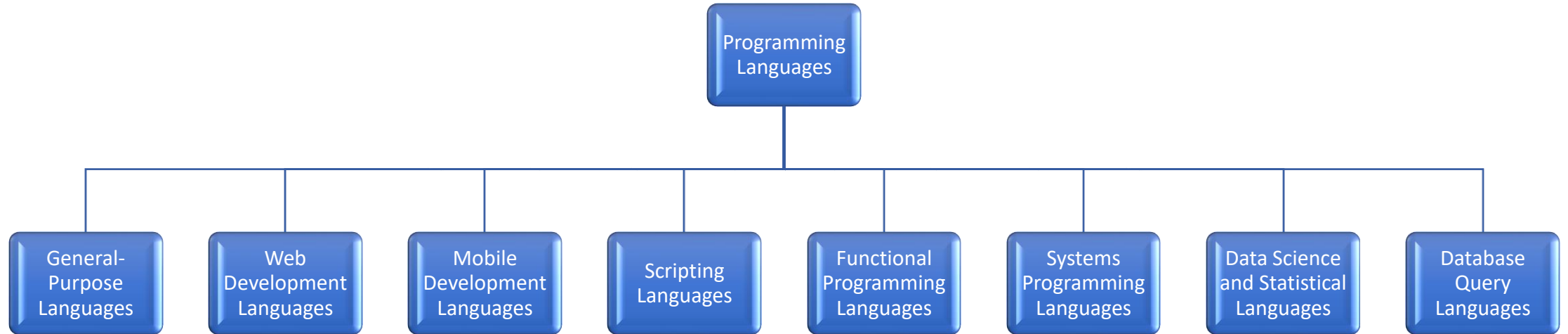
# Code Smells as a Guide

- Code smells are subtle indicators that something might be wrong with the code
- These are not bugs but symptoms of deeper issues that might lead to bugs or increased technical debt if left unresolved.
- Code smells are a guide for refactoring. Common smells include:
  - **Duplicated code:** The same code appears in multiple places.
  - **Long methods:** Methods that do too much, making them hard to understand.
  - **Large classes:** Classes that take on too many responsibilities.
  - **Feature envy:** When a method in one class seems to be more interested in the data of another class.

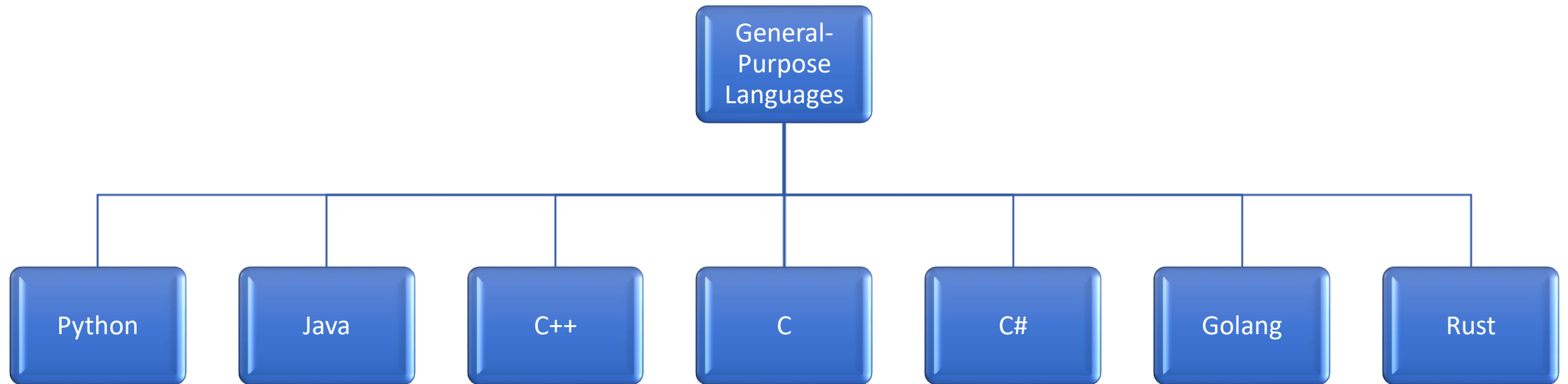
# Don't Refactor Without Reason

- Not all code needs to be refactored. Refactoring should only be done when there is a clear reason, such as improving readability, simplifying future changes, or fixing a code smell.
- Refactoring without a clear goal can waste time, introduce unnecessary complexity, and increase the risk of introducing bugs.
- It's important to balance the need for clean code with the cost of making changes.

# Catalog of Common Programming Languages



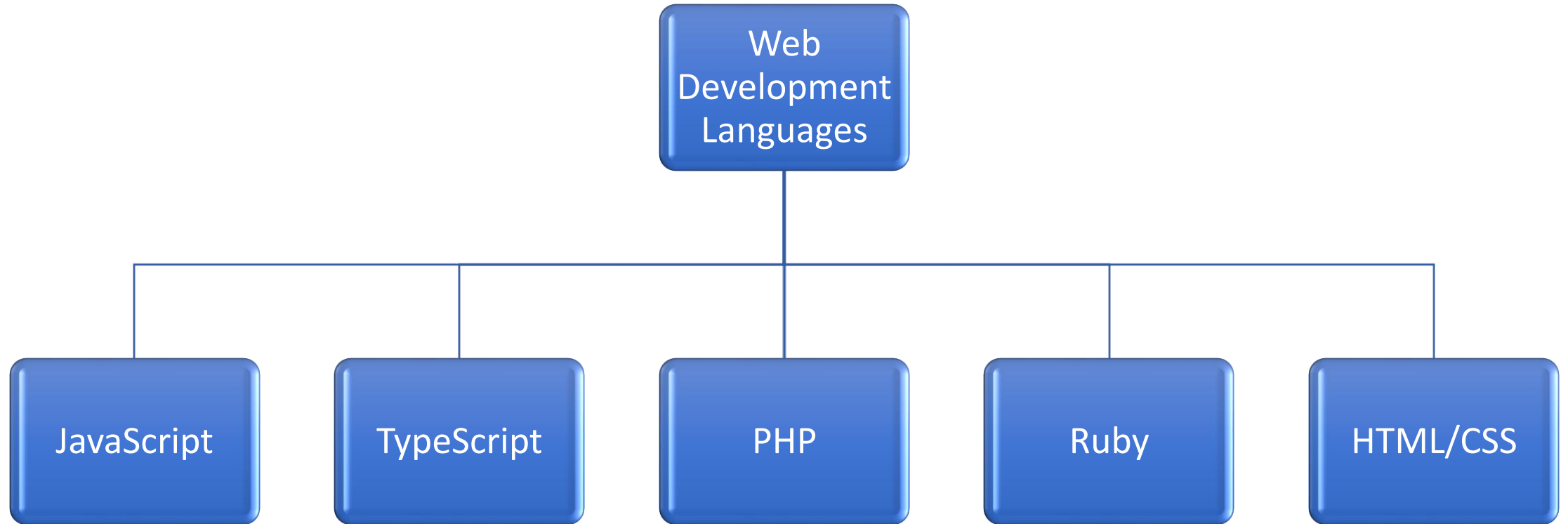
# General-Purpose Languages



# General-Purpose Languages

| Language    | Primary Use                           | Type                       | Notable Features                                      |
|-------------|---------------------------------------|----------------------------|-------------------------------------------------------|
| Python      | Web development, AI/ML, scripting     | High-level, interpreted    | Simple syntax, vast libraries, ideal for data science |
| Java        | Enterprise, Android apps              | Statically typed, compiled | JVM for cross-platform compatibility, OOP-oriented    |
| C++         | Systems, games, performance-critical  | Statically typed, compiled | High performance, fine control over memory management |
| C#          | Desktop apps, game development        | Statically typed, compiled | .NET framework, used in Unity game development        |
| Go (Golang) | Cloud infrastructure, web services    | Statically typed, compiled | Built-in concurrency support (goroutines)             |
| Rust        | Systems programming, embedded systems | Statically typed, compiled | Memory safety without garbage collection              |

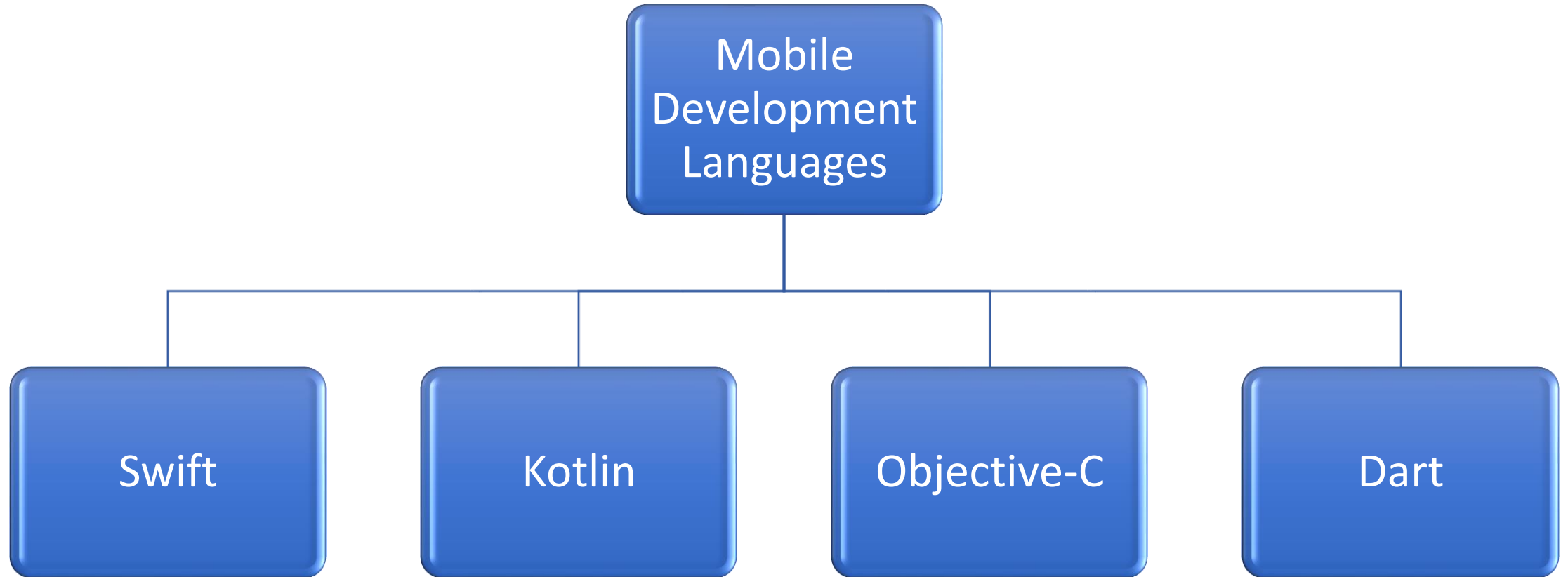
# Web Development Languages



# Web Development Languages

| Language   | Primary Use                       | Type                           | Notable Features                                                      |
|------------|-----------------------------------|--------------------------------|-----------------------------------------------------------------------|
| JavaScript | Client-side scripting, full stack | High-level, dynamically typed  | Ubiquitous in web, both front-end (browser) and server-side (Node.js) |
| TypeScript | Web apps, large-scale JavaScript  | Statically typed, transpiled   | Adds type safety to JavaScript, better for large apps                 |
| PHP        | Server-side web development       | Dynamically typed, interpreted | Popular for content management systems (WordPress)                    |
| Ruby       | Web apps, scripting               | Dynamically typed, interpreted | Ruby on Rails framework, emphasizes simplicity                        |
| HTML/CSS   | Web structure and styling         | Markup, not full languages     | Fundamental for creating web pages                                    |

# Mobile Development Languages

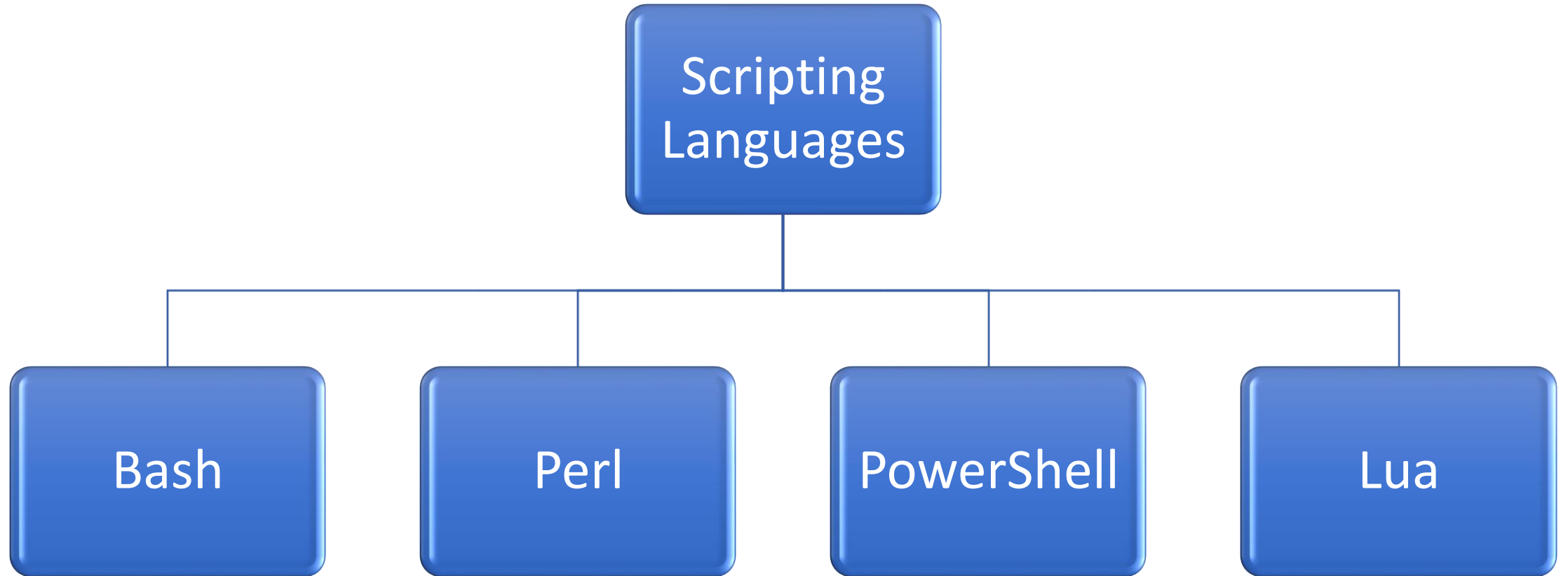




# Mobile Development Languages

| Language    | Primary Use                   | Type                       | Notable Features                                       |
|-------------|-------------------------------|----------------------------|--------------------------------------------------------|
| Swift       | iOS and macOS apps            | Statically typed, compiled | Official language for Apple platforms, fast and safe   |
| Kotlin      | Android apps                  | Statically typed, compiled | Officially supported by Google for Android development |
| Objective-C | Legacy iOS/macOS apps         | Statically typed, compiled | Formerly the main language for Apple development       |
| Dart        | Cross-platform apps (Flutter) | Statically typed, compiled | Used with Flutter for cross-platform mobile apps       |

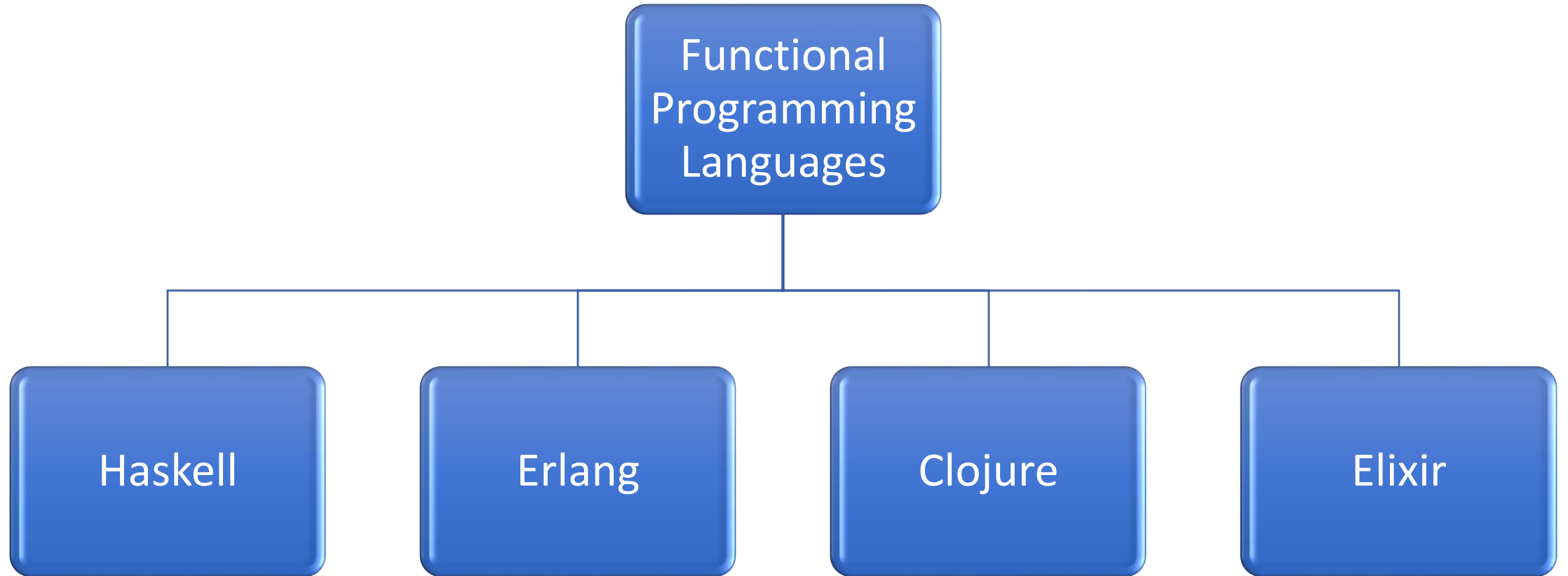
# Scripting Languages



# Scripting Languages

| Language   | Primary Use                      | Type                           | Notable Features                                   |
|------------|----------------------------------|--------------------------------|----------------------------------------------------|
| Bash       | System administration, scripting | Interpreted, dynamically typed | Unix shell scripting, automates system tasks       |
| Perl       | Text processing, web development | Dynamically typed, interpreted | Strong in regex, fast for scripting and automation |
| PowerShell | System administration (Windows)  | Dynamically typed, interpreted | Automates Windows tasks, supports object pipelines |
| Lua        | Game development, scripting      | Dynamically typed, interpreted | Lightweight scripting, often embedded in apps      |

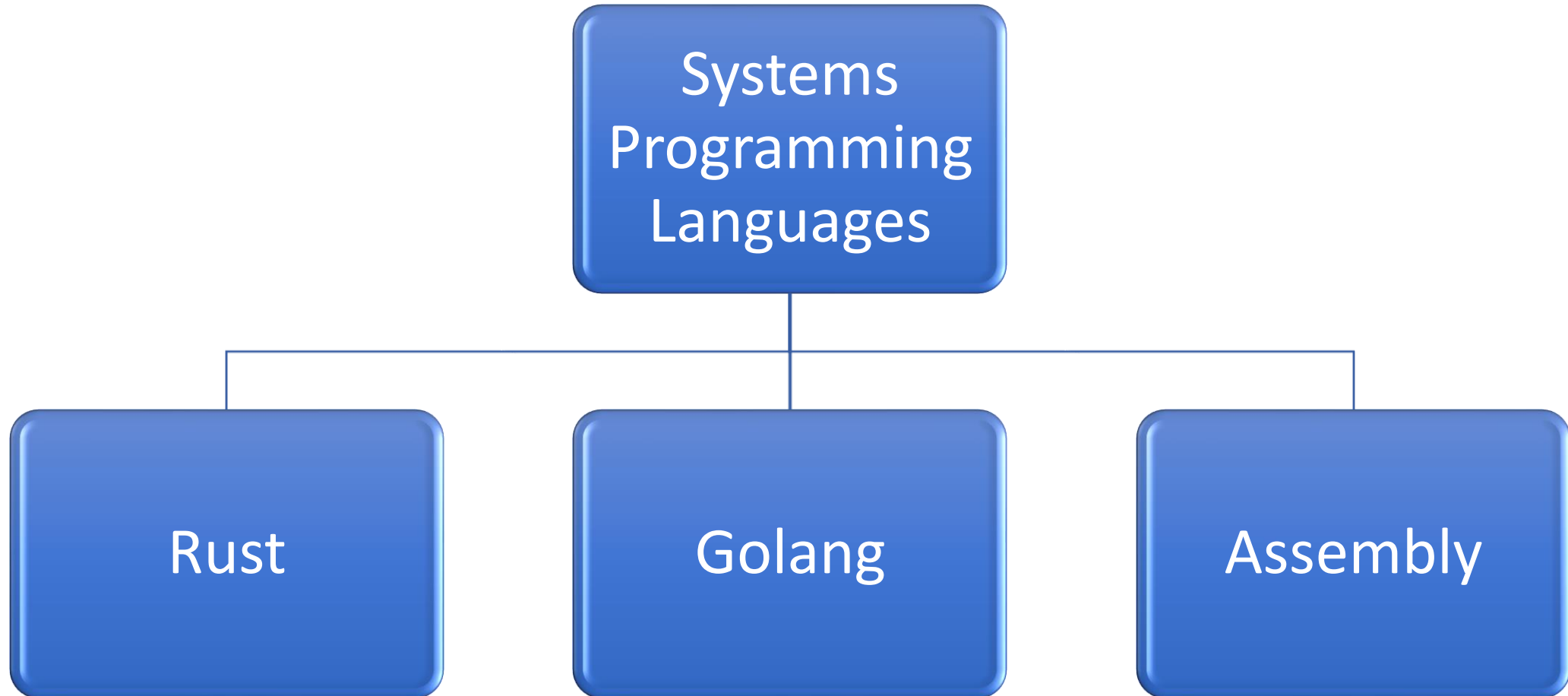
# Functional Programming Languages



# Functional Programming Languages

| Language | Primary Use                           | Type                           | Notable Features                                    |
|----------|---------------------------------------|--------------------------------|-----------------------------------------------------|
| Haskell  | Academic, high-assurance software     | Statically typed, compiled     | Pure functional, lazy evaluation                    |
| Erlang   | Real-time, concurrent systems         | Dynamically typed, interpreted | Designed for telecom systems, high concurrency      |
| Clojure  | Functional web apps, JVM environments | Dynamically typed, compiled    | Lisp dialect, focuses on immutability, runs on JVM  |
| Elixir   | Distributed systems, scalable apps    | Dynamically typed, compiled    | Built on Erlang VM, high concurrency, modern syntax |

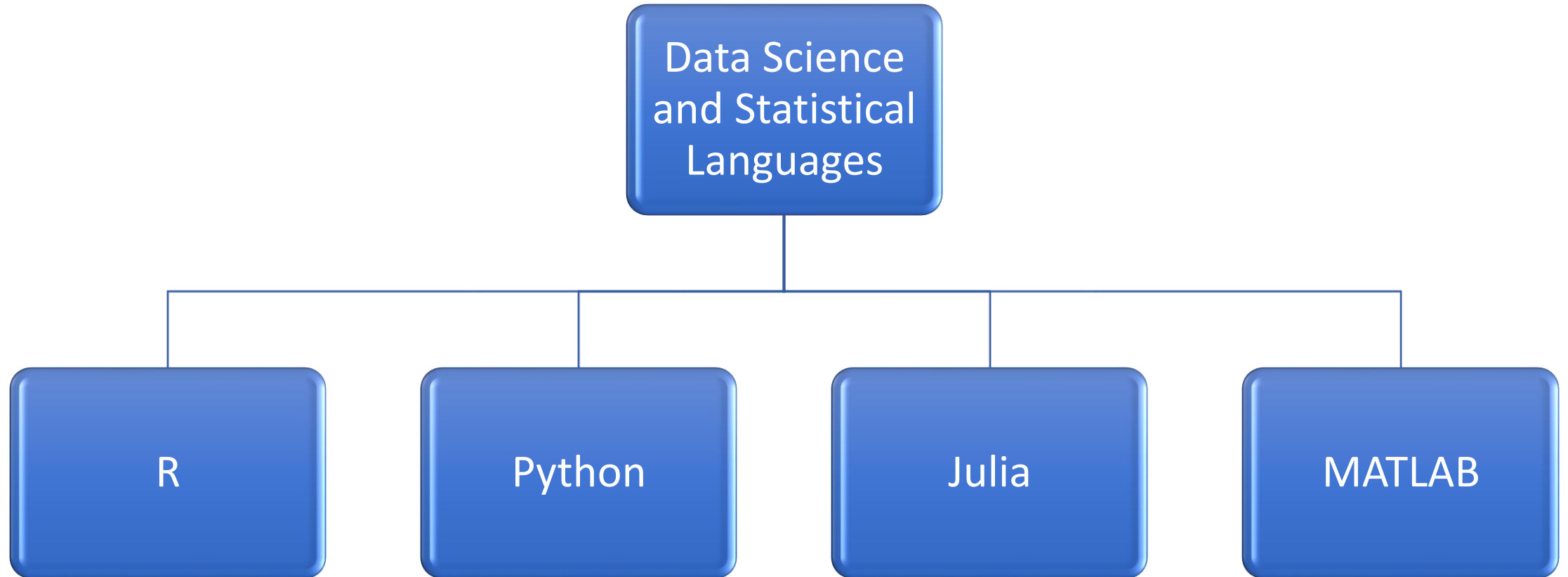
# Systems Programming Languages



# Systems Programming Languages

| Language | Primary Use                                      | Type                        | Notable Features                                       |
|----------|--------------------------------------------------|-----------------------------|--------------------------------------------------------|
| C        | Operating systems, embedded systems              | Statically typed, compiled  | Low-level access to memory, extremely fast             |
| Rust     | Systems, embedded, and performance-critical apps | Statically typed, compiled  | Memory safety without garbage collection               |
| Assembly | Hardware-specific programming                    | Low-level, machine-specific | Direct control of hardware, extremely high performance |

# Data Science and Statistical Languages

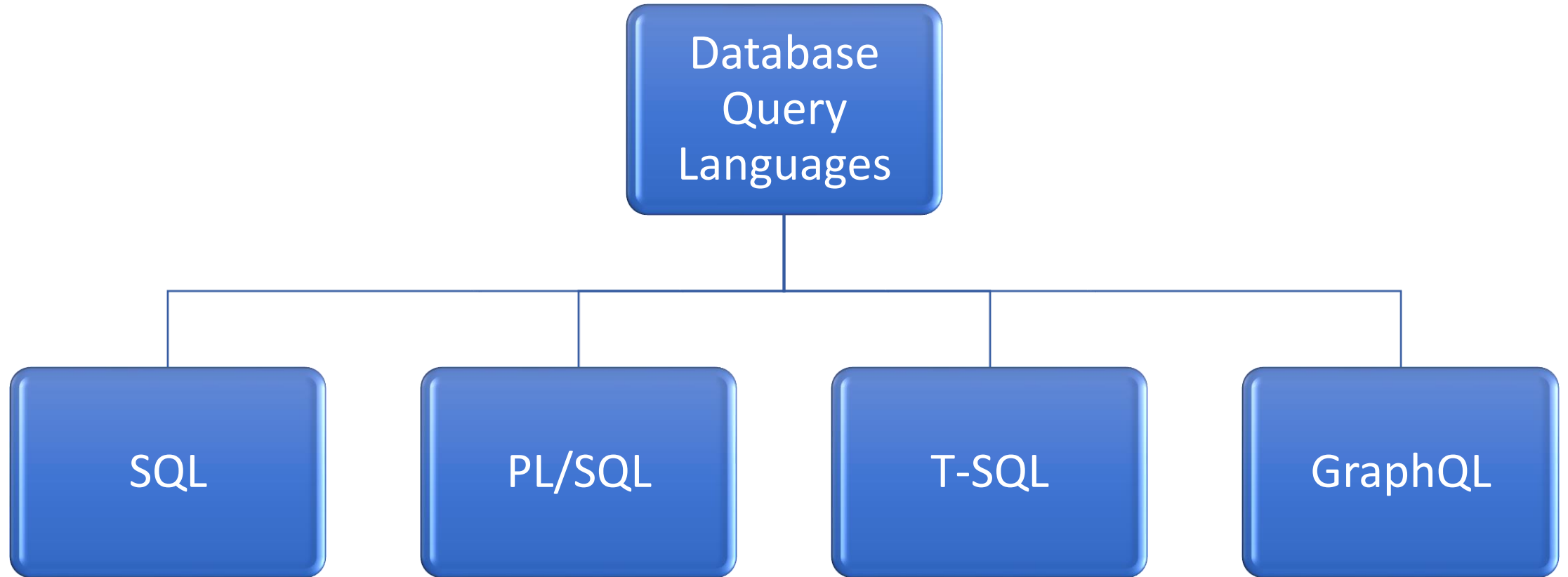




# Data Science and Statistical Languages

| Language | Primary Use                          | Type                           | Notable Features                                            |
|----------|--------------------------------------|--------------------------------|-------------------------------------------------------------|
| R        | Data science, statistical computing  | Dynamically typed, interpreted | Built-in support for statistical analysis, visualization    |
| Python   | Data science, AI/ML, automation      | Dynamically typed, interpreted | Libraries like NumPy, pandas, TensorFlow, and SciPy         |
| Julia    | High-performance numerical computing | Dynamically typed, compiled    | Fast for mathematical operations, used for machine learning |
| MATLAB   | Numerical computing, simulations     | Dynamically typed, interpreted | Strong support for matrix operations and algorithms         |

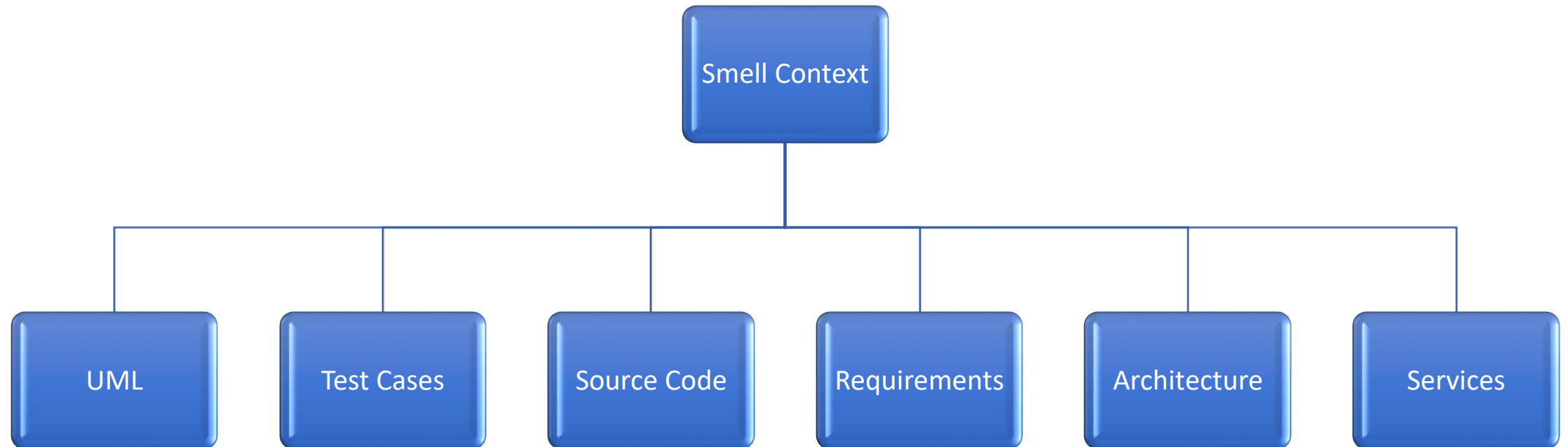
# Database Query Languages



# Database Query Languages

| Language | Primary Use                     | Type                         | Notable Features                                     |
|----------|---------------------------------|------------------------------|------------------------------------------------------|
| SQL      | Database management and queries | Declarative, domain-specific | Standard for relational database operations          |
| PL/SQL   | Oracle database scripting       | Procedural extension of SQL  | Extends SQL with procedural capabilities             |
| T-SQL    | Microsoft SQL Server scripting  | Procedural extension of SQL  | Extends SQL with procedural features, used in MSSQL  |
| GraphQL  | Query language for APIs         | Query language               | Fetches specific data from APIs, alternative to REST |

# Anti-patterns/Code Smells Context





Questions?